lab3

```
function lab3
  clear; clc; close all;
  more off;

  % ============================================================
  % Lab 3 - M/M/1/10 Simulation
  % Questions (1) and (2)
  % Octave-compatible single-file function version
  % File name must be: lab3.m
  % Run with: lab3
  % ============================================================

  SEED = 1;          % same seed in all runs
  mu = 5;            % service rate (customers / min)
  K = 10;            % system capacity
  lambdas = [1, 5, 10];

  % Required (T, B) pairs
  configs = [
      100000,    5000;
      1000000,   20000;
      10000000, 120000
  ];

  % ------------------------------------------------------------
  % DEBUG SECTION
  % ------------------------------------------------------------
  % For question (1), keep DEBUG_TRACE = true to print the first
  % 30 transitions. After verifying correctness, set it to false.
  DEBUG_TRACE = true;
  DEBUG_STEPS = 30;
  TRACE_LAMBDA = 5;
  TRACE_T = DEBUG_STEPS;
  TRACE_B = 0;

  % Sampling step for convergence plots of L
  SAMPLE_EVERY = 1000;

  fprintf('============================================================\n');
  fprintf('DEBUG TRACE - first %d transitions\n', DEBUG_STEPS);
  fprintf('lambda = %g, T = %d, B = %d, seed = %d\n', ...
          TRACE_LAMBDA, TRACE_T, TRACE_B, SEED);
  fprintf('============================================================\n');

  simulate_mm1k_lab3(TRACE_LAMBDA, mu, K, TRACE_T, TRACE_B, ...
                     SEED, DEBUG_TRACE, DEBUG_STEPS, SAMPLE_EVERY);

  fprintf('\nEnd of debug trace.\n');
  fprintf('To disable debugging for the normal executions, set:\n');
  fprintf('DEBUG_TRACE = false\n\n');
```

```

% ------------------------------------------------------------
% NORMAL EXECUTIONS
% ------------------------------------------------------------
results = cell(rows(configs), length(lambdas));

for cfg = 1:rows(configs)
  T = configs(cfg, 1);
  B = configs(cfg, 2);

  for li = 1:length(lambdas)
    lambda = lambdas(li);

    fprintf('============================================================\n');
    fprintf('Running simulation for lambda = %g, T = %d, B = %d\n', ...
            lambda, T, B);
    fprintf('============================================================\n');

    results{cfg, li} = simulate_mm1k_lab3(lambda, mu, K, T, B, ...
                                          SEED, false, DEBUG_STEPS, SAMPLE_EVERY);

    print_results(results{cfg, li});
  endfor

  % -------------------------------------------------------
  % Figure 1 for this (T,B): final ergodic probabilities
  % after removing transient
  % -------------------------------------------------------
  figure(cfg);
  clf;

  for li = 1:length(lambdas)
    subplot(length(lambdas), 1, li);
    bar(0:K, results{cfg, li}.P_B);
    grid on;
    xlim([-0.5, K + 0.5]);
    ylim([0, max(0.01, 1.05 * max(results{cfg, li}.P_B))]);
    title(sprintf(['Ergodic state probabilities after transient removal ', ...
                   '(lambda = %g, T = %d, B = %d)'], ...
                   lambdas(li), T, B));
    xlabel('State');
    ylabel('Probability');
  endfor

  % -------------------------------------------------------
  % Figure 2 for this (T,B): evolution of estimated mean
  % number in system as a function of transitions
  % -------------------------------------------------------
  figure(cfg + rows(configs));
  clf;

  for li = 1:length(lambdas)
    subplot(length(lambdas), 1, li);

```

```
      plot(results{cfg, li}.sample_steps, ...
           results{cfg, li}.L_evolution_all, 'r', 'LineWidth', 1.2);
      hold on;

      if ~isempty(results{cfg, li}.sample_steps_B)
        plot(results{cfg, li}.sample_steps_B, ...
             results{cfg, li}.L_evolution_B, 'b', 'LineWidth', 1.2);
      endif

      yl = ylim();
      line([B, B], yl, 'Color', 'k', 'LineStyle', '--', 'LineWidth', 1.0);

      grid on;
      title(sprintf(['Evolution of estimated mean number in system ', ...
                     '(lambda = %g, T = %d, B = %d)'], ...
                     lambdas(li), T, B));
      xlabel('Number of transitions');
      ylabel('L estimate');
      legend('All transitions', 'After transient', 'B cutoff', 'Location', 'best');
      hold off;
    endfor
  endfor
endfunction


function results = simulate_mm1k_lab3(lambda, mu, K, T, B, seed, ...
                                      debug_trace, debug_steps, sample_every)

  t_start = tic;

  % Reset seed in every run, as required
  rand("seed", seed);

  % System state
  state = 0;
  current_time = 0.0;

  % FIFO storage of accepted customers' arrival times
  arrival_times = [];
  arrival_after_B = [];

  % ----------------------------
  % Statistics - all transitions
  % ----------------------------
  arrival_attempts = 0;
  rejections = 0;
  time_in_state = zeros(1, K + 1);
  total_time = 0.0;
  area_num = 0.0;          % integral of N(t) dt
  sojourn_sum = 0.0;
  served_customers = 0;

  % ----------------------------------
  % Statistics - after removing first B
```

```
% ----------------------------------
arrival_attempts_B = 0;
rejections_B = 0;
time_in_state_B = zeros(1, K + 1);
total_time_B = 0.0;
area_num_B = 0.0;
sojourn_sum_B = 0.0;
served_customers_B = 0;

% -----------------------------
% For convergence / evolution L
% -----------------------------
n_samples = ceil(T / sample_every);

sample_steps = zeros(1, n_samples);
L_evolution_all = zeros(1, n_samples);

sample_steps_B = zeros(1, n_samples);
L_evolution_B = zeros(1, n_samples);

sample_idx = 0;
sample_idx_B = 0;

% -----------------------------
% Main simulation loop
% -----------------------------
for step = 1:T

  % Event selection and total rate
  if state == 0
    is_arrival = 1;
    total_rate = lambda;
  else
    total_rate = lambda + mu;
    is_arrival = (rand() < lambda / (lambda + mu));
  endif

  % Time to next event ~ Exp(total_rate)
  u = rand();
  while u == 0
    u = rand();
  endwhile
  dt = -log(u) / total_rate;
  next_time = current_time + dt;

  % State remains constant during [current_time, next_time)
  time_in_state(state + 1) = time_in_state(state + 1) + dt;
  total_time = total_time + dt;
  area_num = area_num + state * dt;

  if step > B
    time_in_state_B(state + 1) = time_in_state_B(state + 1) + dt;
    total_time_B = total_time_B + dt;
    area_num_B = area_num_B + state * dt;
```

```
    endif

    % Process the event at next_time
    if is_arrival

      arrival_attempts = arrival_attempts + 1;
      if step > B
        arrival_attempts_B = arrival_attempts_B + 1;
      endif

      if state < K
        % Accepted arrival
        state_new = state + 1;

        arrival_times(end + 1) = next_time;
        arrival_after_B(end + 1) = (step > B);

        accepted_text = 'accepted';
      else
        % Rejected arrival
        state_new = state;
        rejections = rejections + 1;

        if step > B
          rejections_B = rejections_B + 1;
        endif

        accepted_text = 'rejected';
      endif

    else
      % Departure
      state_new = state - 1;

      customer_arrival_time = arrival_times(1);
      customer_after_B = arrival_after_B(1);

      arrival_times(1) = [];
      arrival_after_B(1) = [];

      sojourn = next_time - customer_arrival_time;

      sojourn_sum = sojourn_sum + sojourn;
      served_customers = served_customers + 1;

      % For post-transient W, keep only customers accepted after B
      if customer_after_B
        sojourn_sum_B = sojourn_sum_B + sojourn;
        served_customers_B = served_customers_B + 1;
      endif
    endif

    % Debug print for the first 30 transitions
    if debug_trace && step <= debug_steps
```

```
      if is_arrival
        printf(['Step %2d | state=%2d | event=arrival (%s) | ', ...
                'arrivals=%2d | new state=%2d\n'], ...
                step, state, accepted_text, arrival_attempts, state_new);
      else
        printf(['Step %2d | state=%2d | event=departure        | ', ...
                'arrivals=%2d | new state=%2d\n'], ...
                step, state, arrival_attempts, state_new);
      endif
    endif

    % Move to next state / time
    state = state_new;
    current_time = next_time;

    % Sample convergence curve of L
    if mod(step, sample_every) == 0 || step == T
      sample_idx = sample_idx + 1;
      sample_steps(sample_idx) = step;
      L_evolution_all(sample_idx) = area_num / total_time;

      if total_time_B > 0
        sample_idx_B = sample_idx_B + 1;
        sample_steps_B(sample_idx_B) = step;
        L_evolution_B(sample_idx_B) = area_num_B / total_time_B;
      endif
    endif
  endfor

  % ----------------------------
  % Final metrics
  % ----------------------------
  results.lambda = lambda;
  results.mu = mu;
  results.K = K;
  results.T = T;
  results.B = B;
  results.runtime_sec = toc(t_start);

  % All transitions
  results.P = time_in_state / total_time;
  results.L = area_num / total_time;
  results.P_rej = rejections / arrival_attempts;

  if served_customers > 0
    results.W = sojourn_sum / served_customers;
  else
    results.W = NaN;
  endif

  % After transient
  if total_time_B > 0
    results.P_B = time_in_state_B / total_time_B;
    results.L_B = area_num_B / total_time_B;
```

```
  else
    results.P_B = NaN(1, K + 1);
    results.L_B = NaN;
  endif

  if arrival_attempts_B > 0
    results.P_rej_B = rejections_B / arrival_attempts_B;
  else
    results.P_rej_B = NaN;
  endif

  if served_customers_B > 0
    results.W_B = sojourn_sum_B / served_customers_B;
  else
    results.W_B = NaN;
  endif

  % Curves
  results.sample_steps = sample_steps(1:sample_idx);
  results.L_evolution_all = L_evolution_all(1:sample_idx);

  if sample_idx_B > 0
    results.sample_steps_B = sample_steps_B(1:sample_idx_B);
    results.L_evolution_B = L_evolution_B(1:sample_idx_B);
  else
    results.sample_steps_B = [];
    results.L_evolution_B = [];
  endif
endfunction


function print_results(results)
  fprintf('lambda = %g, mu = %g, K = %d, T = %d, B = %d\n', ...
          results.lambda, results.mu, results.K, results.T, results.B);
  fprintf('Runtime = %.4f sec\n\n', results.runtime_sec);

  fprintf('--- Using all transitions ---\n');
  fprintf('Mean number in system L       = %.10f\n', results.L);
  fprintf('Rejection probability P_rej   = %.10f\n', results.P_rej);
  fprintf('Mean time in system W         = %.10f\n', results.W);
  fprintf('State probabilities P         = [');
  fprintf(' %.8f', results.P);
  fprintf(' ]\n\n');

  fprintf('--- Ignoring first B transitions ---\n');
  fprintf('Mean number in system L_B     = %.10f\n', results.L_B);
  fprintf('Rejection probability P_rej_B = %.10f\n', results.P_rej_B);
  fprintf('Mean time in system W_B       = %.10f\n', results.W_B);
  fprintf('State probabilities P_B       = [');
  fprintf(' %.8f', results.P_B);
  fprintf(' ]\n\n');

  fprintf('Relative difference in L      = %.6f %%\n', ...
          100 * abs(results.L_B - results.L) / max(abs(results.L), eps));
```

```
  fprintf('Relative difference in P_rej  = %.6f %%\n', ...
          100 * abs(results.P_rej_B - results.P_rej) / max(abs(results.P_rej), eps));
  fprintf('Relative difference in W      = %.6f %%\n\n', ...
          100 * abs(results.W_B - results.W) / max(abs(results.W), eps));
endfunction
```